NextAIRE

NextAIRE

Next Generation AI Researchers for Air Quality Excellence

Funded by
the European Union

NextAIRE

Data Management for Low-cost Sensors
Module 1: Data modeling and Databases

Funded by
the European Union

NextAIRE

Consortium

Ascalia

diadikasia
BRIGHT GROUP MEMBER

smartsense

Megasense

CENTER FOR PHYSICAL SCIENCES AND TECHNOLOGY

INSTITUTE FOR ANTHROPOLOGICAL RESEARCH

theLisbonCouncil
think tank for the 21st century

INAR
Institute for Atmospheric and Earth System Research

imi

msensis

vito

KNOW Center

DHMZ

UNIVERSITAS UNIVERZITET SARAJEVU STUDIORUM SARAJEVOENSIS

DEMOKRITOS

FERIT
FACULTY OF ELECTRICAL ENGINEERING, COMPUTER SCIENCE AND INFORMATION TECHNOLOGY OSIJEK

Funded by
the European Union

NextAIRE

Agenda

- Introduction to Databases
- Conceptual Data Model
- Relational Data Model
- Structured Query Language

Funded by
the European Union

NextAIRE

Short introduction to databases

Funded by
the European Union

History of data storage and management

- Essential for humans to store and retrieve information/data to make informed decisions
  - Should be as efficient as possible
  - Categorization, no duplicates, relations, etc …

- Manual records of Sumerians are an early example of storing data (4000 BC using clay)

[archive.org]

Funded by
the European Union

History of data storage and management

- From clay, we moved to papyrus, parchment to paper

Funded by
the European Union

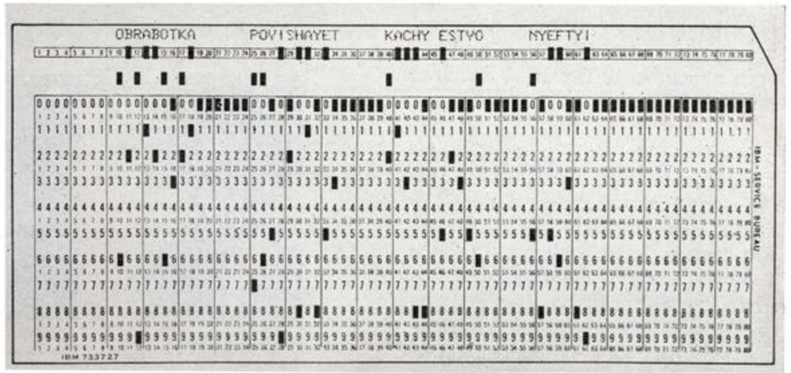
History of data storage and management

- From clay, we moved to papyrus, parchment to paper

- Data management in libraries → Dewey Decimal System
  - Developed around 1876 and still used today in 135 countries
  - Organizes books in libraries based on subjects
  - 3-digit numbers for main classes (e.g., 300 is social sciences)
  - Fractional decimals for detailed classes (e.g., 323 is civil and political rights)

- Punch cards (1900 – 1955)

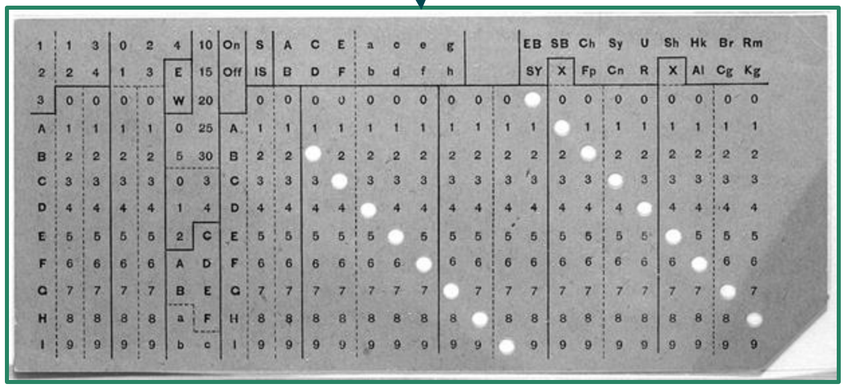
- 1960 marks the start of computerized databases



Funded by
the European Union

Computerized Databases

- **Flat files**
  - Flat → no connection with other files/records
  - E.g. Hollerit's punch cards database
  - Stored as sequential **files retrieval is slow**
  - Example: .csv file

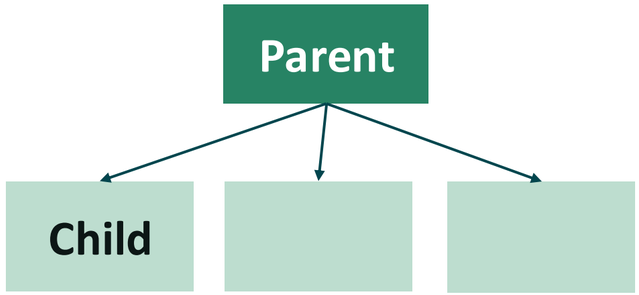


Funded by
the European Union

Computerized Databases

- **Flat files**
  - Flat → no connection with other files/records
  - Stored as sequential files → retrieval is slow
  - Example: .csv file

- **Hierarchical model**
  - Tree of records
  - E.g., IBM Information Management System
  - Each child can only have one parent



Funded by
the European Union

Relational Databases (1970/80s - now)

School Table

ID	Name
S001	University of Technology
S002	University of Applied Science

Reference

Student Table

School ID	ID	Name	DOB
S001	UT-1000	Tommy	05/06/1995
S001	UT-1000	Better	16/04/1995
S002	UAS-1000	Linda	02/09/1995
S002	UAS-1000	Jonathan	22/06/1995

Tuple

Attribute

Relation (set of tuples with attributes, often the whole table)

Funded by
the European Union

Relational Databases (1970/80s - now)

- Organize a body of data into simple tables of related information
- There are no pointers to maintain (reference)
- Tables connected only by having matching data field (keys)
- Easy to access, merge and change data
- Independence between data scheme and physical storage
- SQL (structured query language)

Funded by
the European Union

Recommended DBMS: PostgreSQL

- Open-source and free relational database management system
- Easy to install, e.g., (https://www.postgresql.org/download/)
  - See also Tutorial pdf in NextAIRE learning platform
- Nice GUI (pgAdmin4)
- Good support and user community

[postgresql.org]

Funded by
the European Union

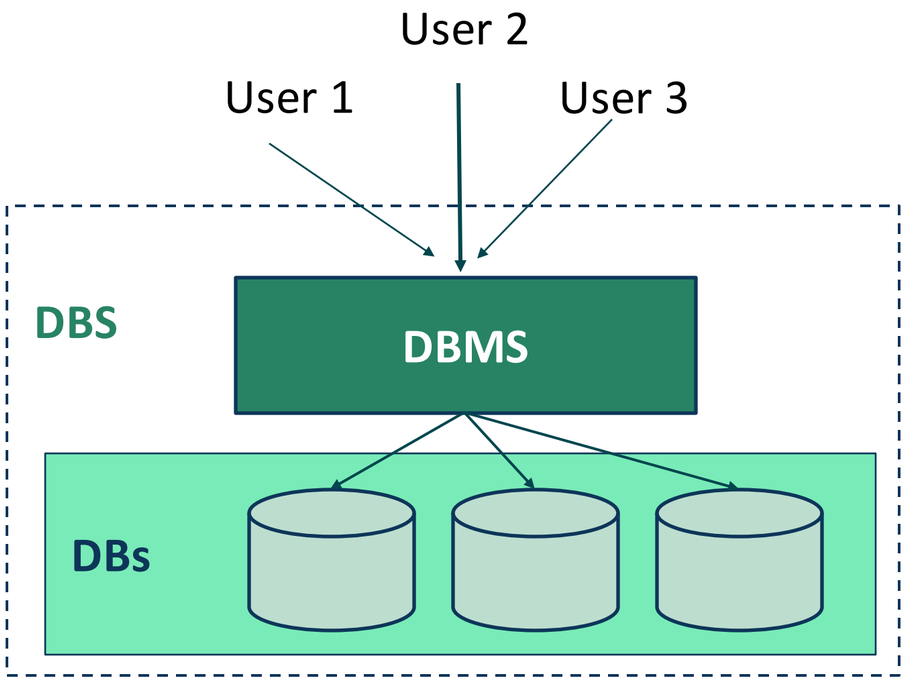
NextAIRE

What is a database?

Funded by
the European Union

Definition of a database

- Database system (DBS): DBMS + DBs
- DBMS: Database Management System (Software to handle DBs)
- DBs: Database (data/metadata collection to describe something)
- Note: DB also a short for DBS/DBMS



Funded by
the European Union

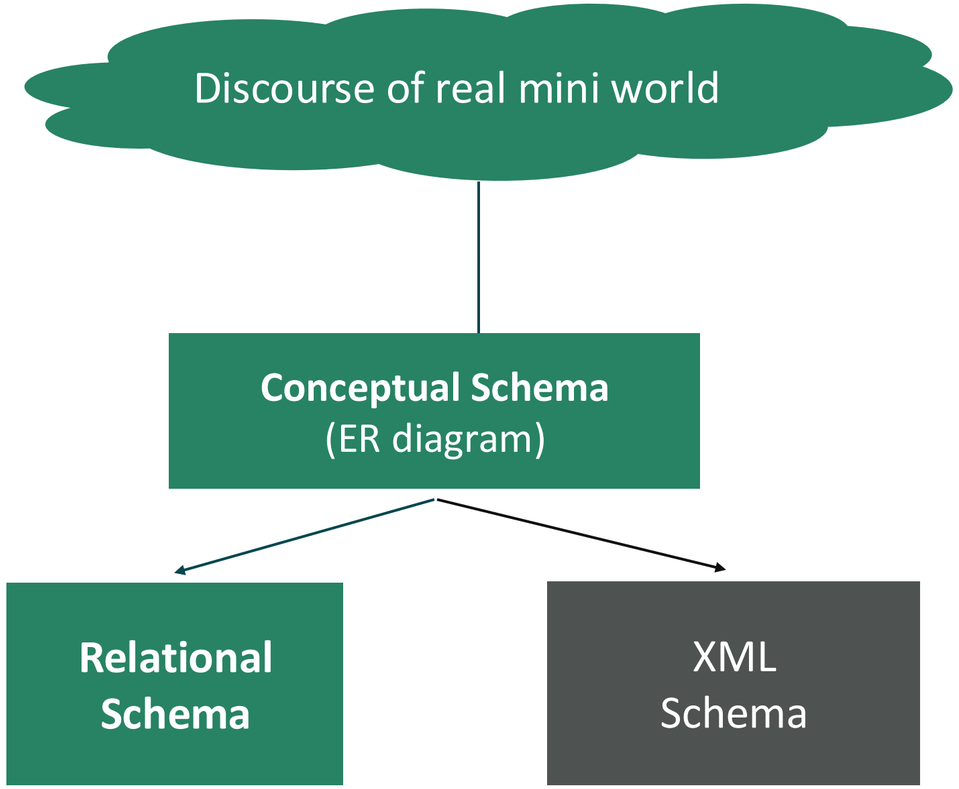
NextAIRE

Database Design

Funded by
the European Union

Data modeling

- Concept for describing data objects and their relationships
- Schema: Description (structure, semantics) of specific data collection



Manual Modeling

Funded by the European Union

Data models

- **Conceptual Data Models**
  - **Entity-Relationship Model (ERM)**, focus on data, ~1975
- **Logical Data Models**
  - **Relational data model**
  - Document (XML, JSON)
  - Time series

Funded by
the European Union

Databases design cycles

- **Requirements engineering**
  - Collect and analyze data and application requirements
  - Specification documents
- **Conceptual Design**
  - Model data semantics and structure, independent of logical data model
  - ER model/ diagram
- **Logical Design**
  - Model data with implementation details of concrete data model (e.g., relational schema)
- **Physical Design**
  - Account for deployment environment and performance requirements

Funded by
the European Union

NextAIRE

Engineering Requirements

Funded by
the European Union

Engineering Requirements

Collected Information

- Stations have a name and a location. They can host several sensors.
- Sensors have a SensorID, type, measured variable, and start date.
- Each sensor belongs to one station, while each station can contain many sensors.
- Campaigns have an ID, start date, and end date.
- Sensors can be used in monitoring campaigns.
- A sensor can be used in several campaigns, and a campaign can use several sensors.
- Some sensors can also validate campaigns.
- Each campaign is validated by one sensor, while one sensor can validate several campaigns.

Funded by
the European Union

NextAIRE

Conceptual Design

Funded by
the European Union

NextAIRE

Entity-relationship model/diagram

Funded by
the European Union

ER Diagram - Chen notation

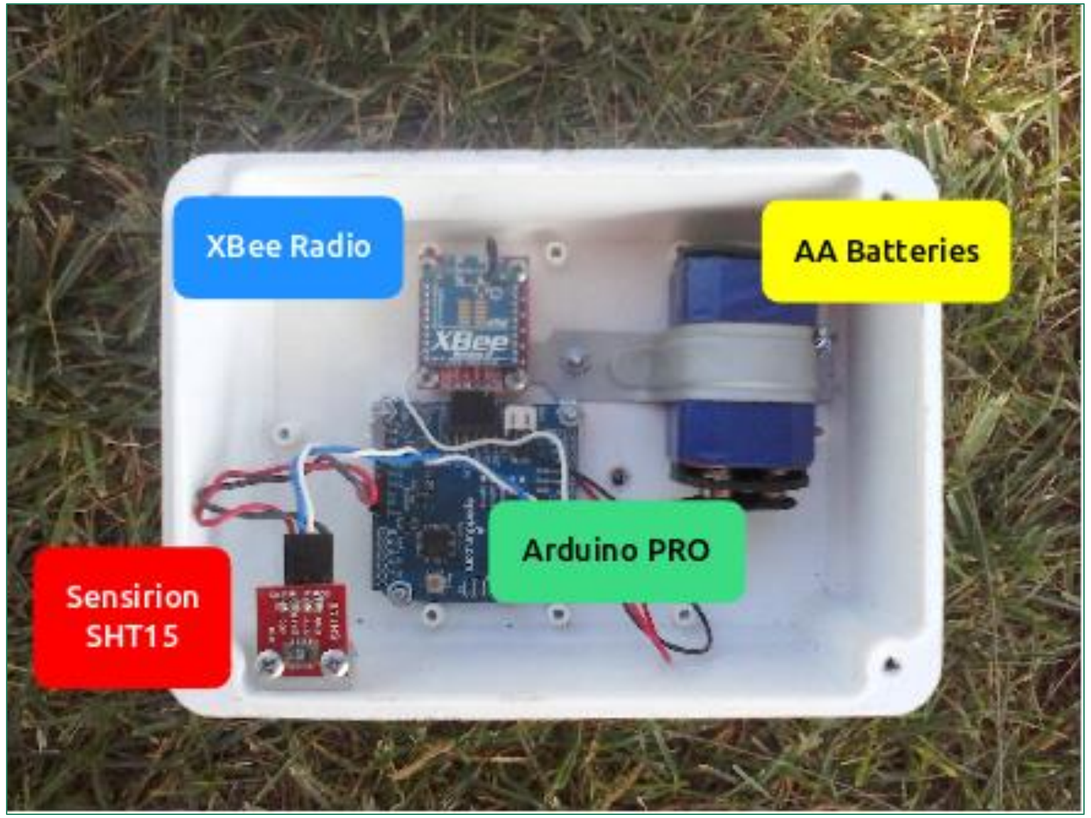
Entity (noun)

- Entities are objects of the real world
- Rectangular shape + capitalized

Funded by
the European Union

Entity Example: Sensor

Attributes	Value
sensor_id	SEN-001
name	Outdoor_weather
sensor_type	Humidity and temperature
sensor_model	Sensirion SHT15
channel_1	Relative humidity
channel_2	Temperature
humidity_range	0 – 100 %RH
humidity_accuracy	±2 %RH
temperature_range	-40 °C to 123 °C
temperature_accuracy	±0.3 °C



Funded by
the European Union

ER Diagram - Chen notation

Entity (noun)

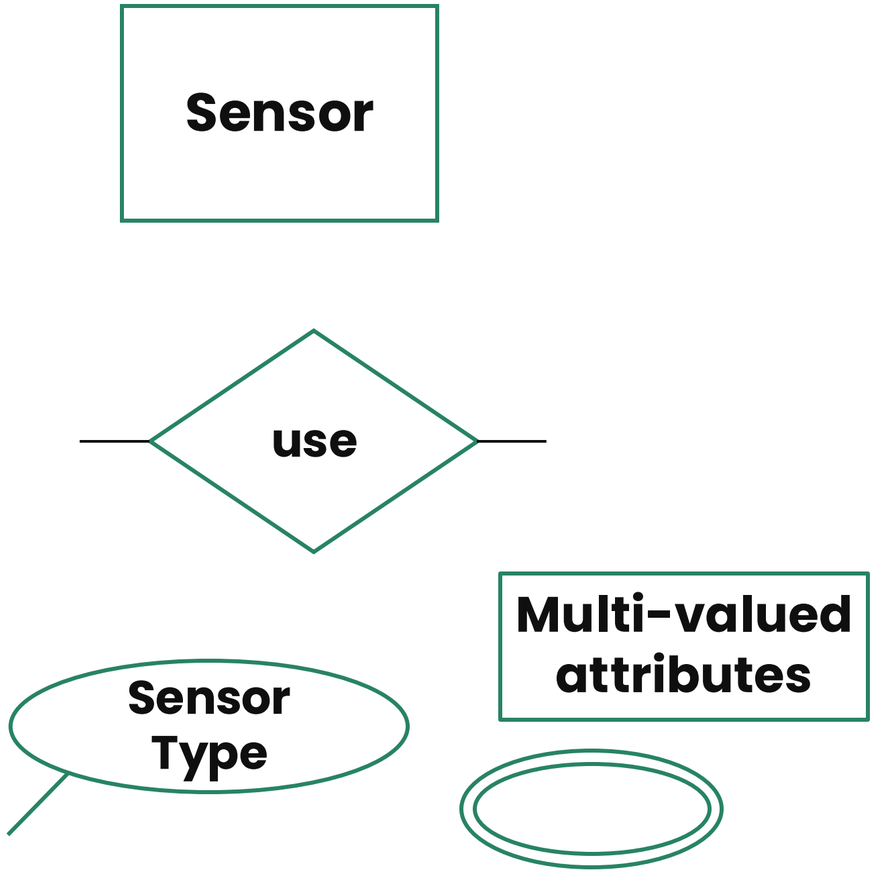
- Entities are objects of the real world
- An entity instance is a concrete instantiation of an entity
- Rectangular shape + capitalized

Relationship (verb)

- Relationships are concrete associations of entities
- Diamond shape + non-capitalized

Attribute

- Entities or relationships are characterized by attribute-value pairs
- **Round shape + capitalized**
- Multi-valued attributes (e.g., phone numbers)

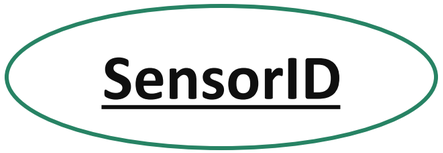


Funded by the European Union

ER Diagram - Chen notation

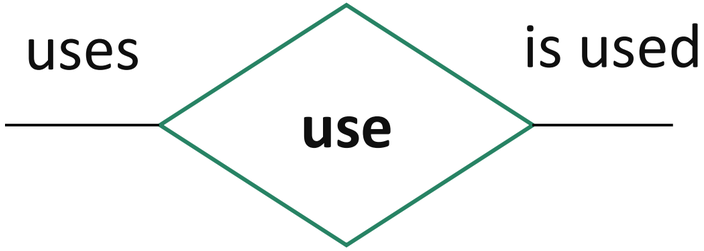
Keys

- Attributes that uniquely identify an entity
- Every entity type must have such a key
- Natural or surrogate (artificial) keys



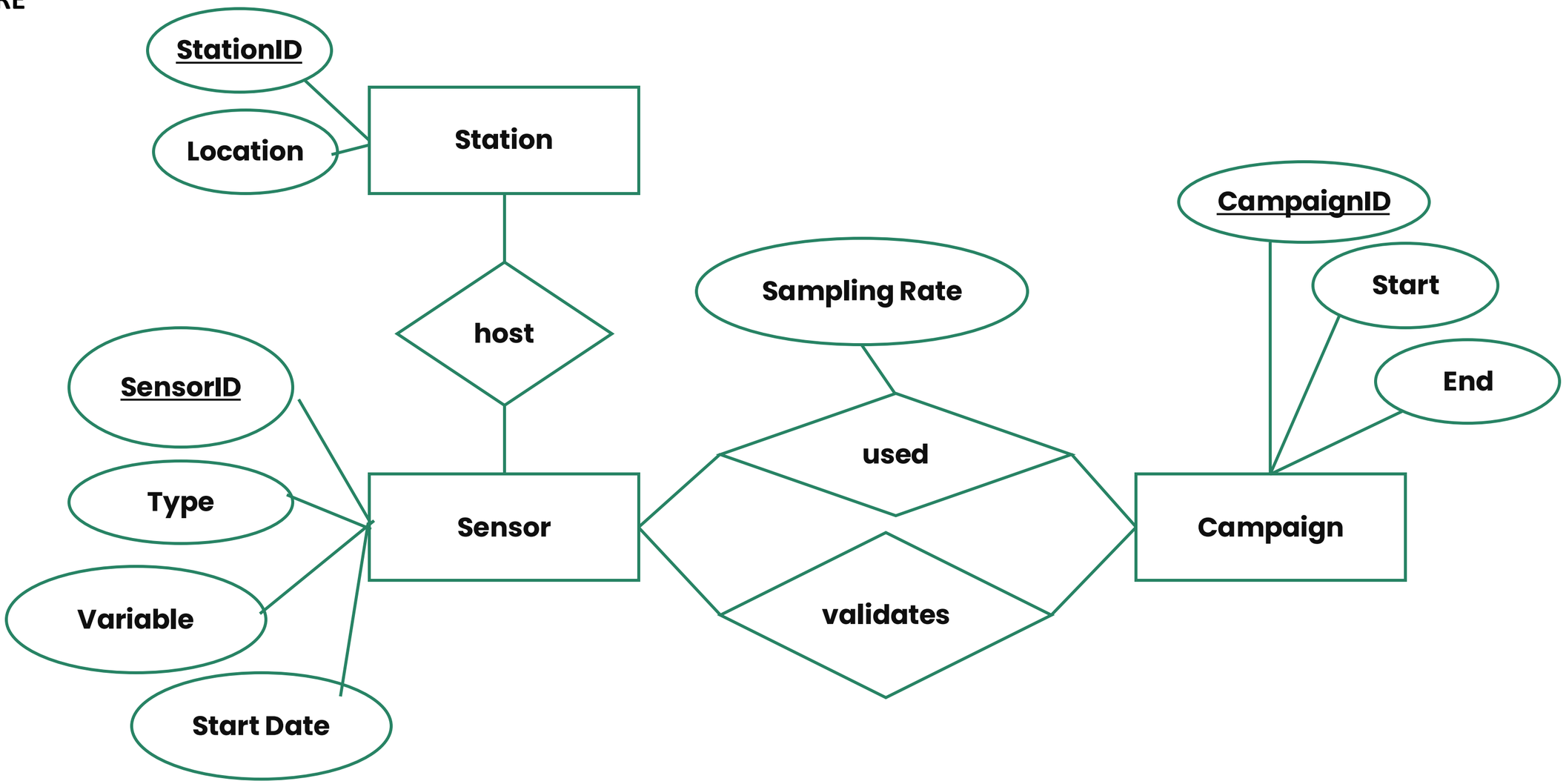
Role

- Optional description of relationship types
- E.g., an sensor is used in a project & a project uses sensors



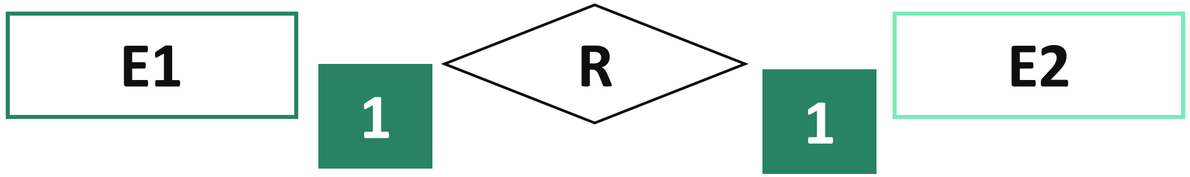
Funded by
the European Union

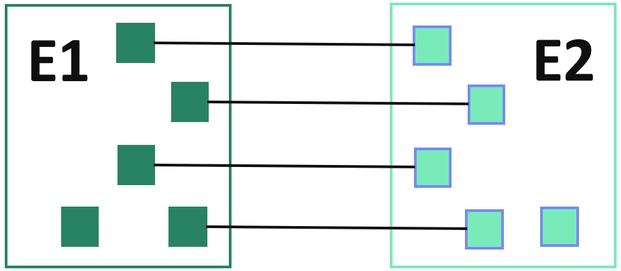
ER Diagram – A sensor database



Funded by
the European Union

ER Diagram – Multiplicity/Cardinality



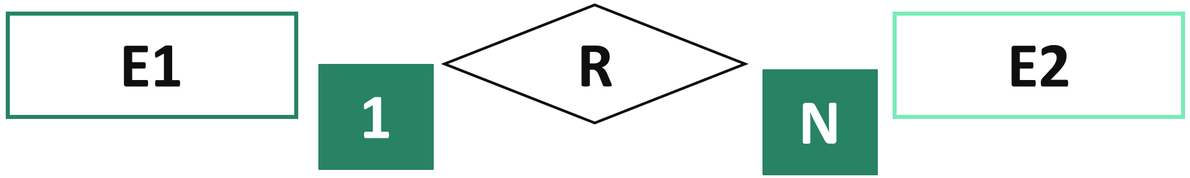


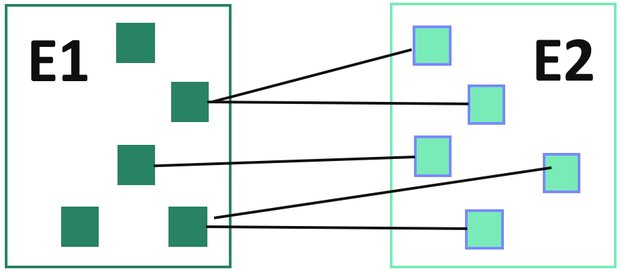
1:1 (one-to-one)

- Each Sensor relates to at most one Enclosure
- Each Enclosure relates to at most one Sensor
- 1 sensor can be installed in 1 enclosure and 1 enclosure can contain 1 sensor

Funded by
the European Union

ER Diagram – Multiplicity/Cardinality



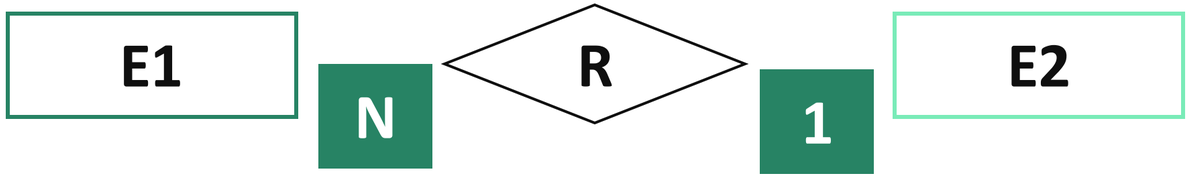


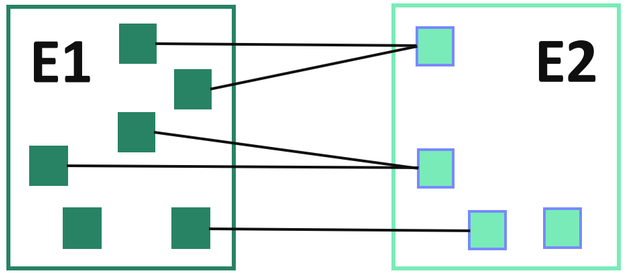
1:N (one-to-many)

- Each Station relates to many Sensors (0, 1, … N)
- Each Sensor relates to at most one Station
- E.g., 1 station can host N sensors but 1 sensor can only be installed at 1 station

Funded by
the European Union

ER Diagram – Multiplicity/Cardinality



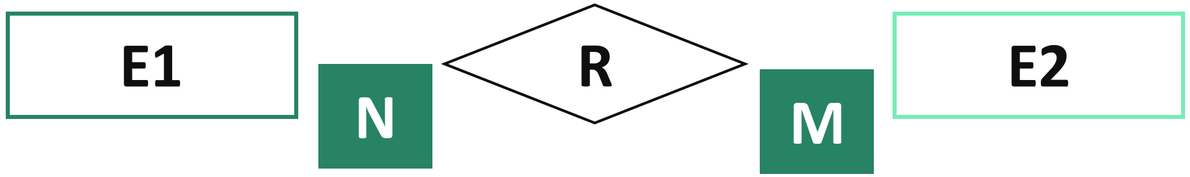


N:1 (many-to-one)

- Each Sensor relates to many Measurements (0, 1, … N)
- Each Measurement relates to at most one Sensor
- E.g., 1 sensor can record N measurements but 1 measurement can be recorded by just 1 sensor

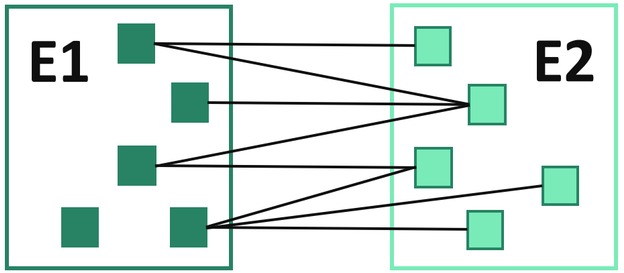
Funded by
the European Union

ER Diagram – Multiplicity/Cardinality



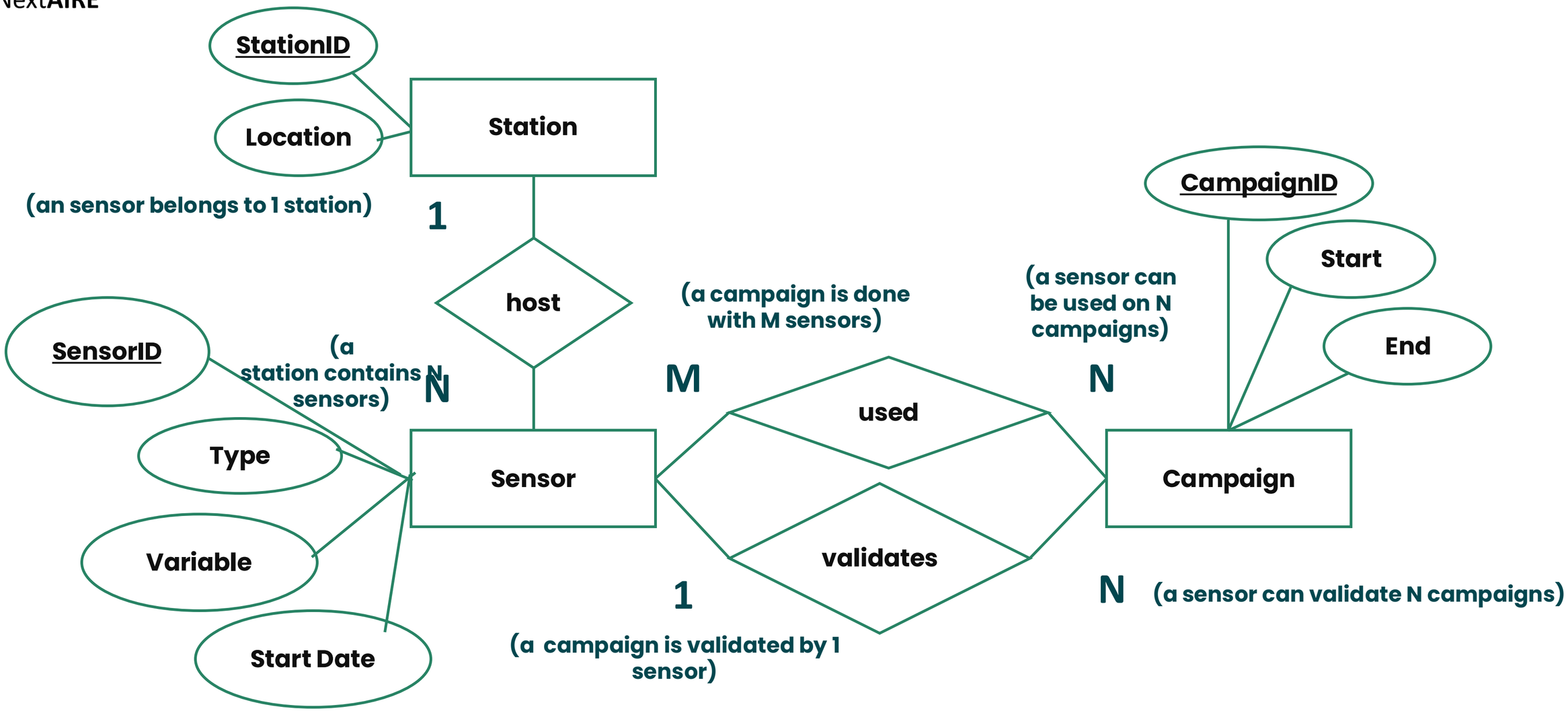
N:M (many-to-many)

- Each Sensor relates to many Campaigns (0, 1, … M)
- Each Campaign relates to many Sensors (0, 1, … N)
- E.g., 1 sensor can be used in M campaigns and 1 campaign can use N sensors



Funded by
the European Union

ER Diagram – Multiplicity/Cardinality



Funded by
the European Union

NextAIRE

Logical Design

Funded by
the European Union

NextAIRE

Relational Data Model

Funded by
the European Union

Relational model

- Represent data as a **collection of tables**

Funded by
the European Union

Relational model

- Represents a table (precisely a part of a table)
- Attributes represent the columns headers
- Tuples represent the rows
- Domain (value domain) represents a data type and set of values allowed for an attribute

My First Table

A1	A2	A3
3	7	T
1	2	T
3	4	F
1	7	T

Tuples

Attribute

Funded by
the European Union

Data types in postgreSQL

Numeric type	Description
integer	4 bytes. Whole numbers from -2,147,483,648 to +2,147,483,647.
smallint	2 bytes. Smaller whole numbers.
bigint	8 bytes. Very large whole numbers.
serial	4 bytes. Auto-incrementing integer, useful for keys such as IDs.
real	4 bytes. Real values with about 6 decimal digits of precision.
double precision	8 bytes. Real values with about 15 decimal digits of precision.
numeric(precision, scale)	Exact decimal values. Example: 24.345 has precision 5 and scale 3.
numeric	Variable size when no precision or scale is provided.

Funded by
the European Union

NextAIRE

Data types in postgreSQL

Character type	Description
char(n)	Stores strings with a fixed length of n. Example: char(5) allows 'hello'. Longer strings cause an error. Shorter strings are padded with spaces.
char	Same as char(1). Stores one character.
varchar(n)	Stores strings with a maximum length of n. Longer strings cause an error. Shorter strings are not padded with spaces.
varchar	Stores strings of any length.
text	Stores strings of any length. In PostgreSQL, it is similar to varchar without a length limit.

Funded by
the European Union

Data types in postgreSQL

Date/time type	Description
date	Stores a calendar date. Input is flexible, for example '1999-01-08' or '1999-Jan-08'. Common format: yyyy-MM-dd.
time	Stores a time of day, for example '04:05:06'. Common format: hh:mm:ss.
time with time zone	Stores a time of day with a time zone offset, for example '04:05:06+02'.
timestamp	Stores date and time together, for example '1999-01-08 04:05:06'. Common format: yyyy-MM-dd hh:mm:ss.
timestamp with timezone /timestamptz	Stores date and time with a time zone offset, for example '1999-01-08 04:05:06+02'.
to_timestamp()	Converts Unix timestamps into timestamp values. Unix timestamps count seconds since 1970-01-01.

Funded by
the European Union

Data types in postgreSQL

Other types	**Description**
Boolean	For storing binary states true / false or 1 / 0 is possible
NULL	This is not a type but used to specify missing values for all types
XML / JSON	To store XML or JSON strings

Full list: https://www.postgresql.org/docs/current/datatype.html

Funded by
the European Union

Domain

Domain (value domain) represents a **data type** and defines the **set of values allowed** for an attribute

Examples:

Age

- Can be represented with **Integer**
- 1,2,3,....,100

Name

- Can be represented as a **String** (array of characters)
- 'Jane Doe', 'John Doe'

Date of Birth

- Can be represented as a **String** or **Date**
- Can have **constraints** (**Date of Birth cannot be beyond the current date**)
- '21-08-1995', '02-07-2041'

A1 INT	A2 INT	A3 BOOL
3	7	T
1	2	T
3	4	F
1	7	T

Funded by
the European Union

Primary Key

Primary Key: Minimal set of attributes X that uniquely identifies tuples in a relation R

#1 Unique: No other tuple with same value

#2 Defined: Not null

#3 Minimal: No attribute can be removed

sensor_id	type	variable	StationID
1	Sensirion SHT15	Temperature and humidity	GRZ01

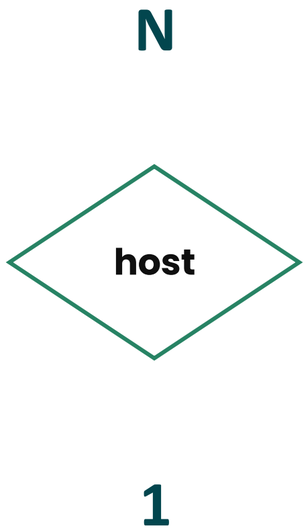
Funded by
the European Union

Foreign Key

Foreign Key: Reference to a primary key in another relation

sensor_id	type	variable	StationID
1	Sensirion SHT15	Temperature and humidity	GRZ01

StationID	Location
GRZ01	Austria



Funded by
the European Union

Referential Integrity Constraints

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

sensor_id	type	variable	StationID
1	Sensirion SHT15	Temperature and humidity	GRZ01
2	DHT22	Temperature and Humidity	MDZ02
3	DHT22	Temperature and Humidity	GRZ01
4	Sensirion SHT15	Temperature and Humidity	GRZ02

Funded by
the European Union

Referential Integrity Constraints

Foreign Keys:

- Reference to a **primary key** in **another relation**
- **Referential integrity:** FK **need to reference existing** tuples or **NULL**

Enforcing Referential Integrity

- **Error** (default)
- **Propagation** on request
  - E.g., for existential dependence
- **Set NULL** on request
  - E.g., for independent entities

```
DELETE FROM Stations WHERE Station ID=GRZ01
```

Funded by
the European Union

Constraints

Domain/semantic constrains:

- Value constraints of individual attributes (e.g StationID lenght = 5 characters)
- **CHECK:** Value ranges or enumerated valid values (e.g humidity between 0 and 100)

- **UNIQUE Constraints**
  - Enforce uniqueness of non-primary key attribute
- **NOT NULL Constraints**
  - Enforce known / existing values
- **DEFAULT Constraints**
  - Sets a default value when created

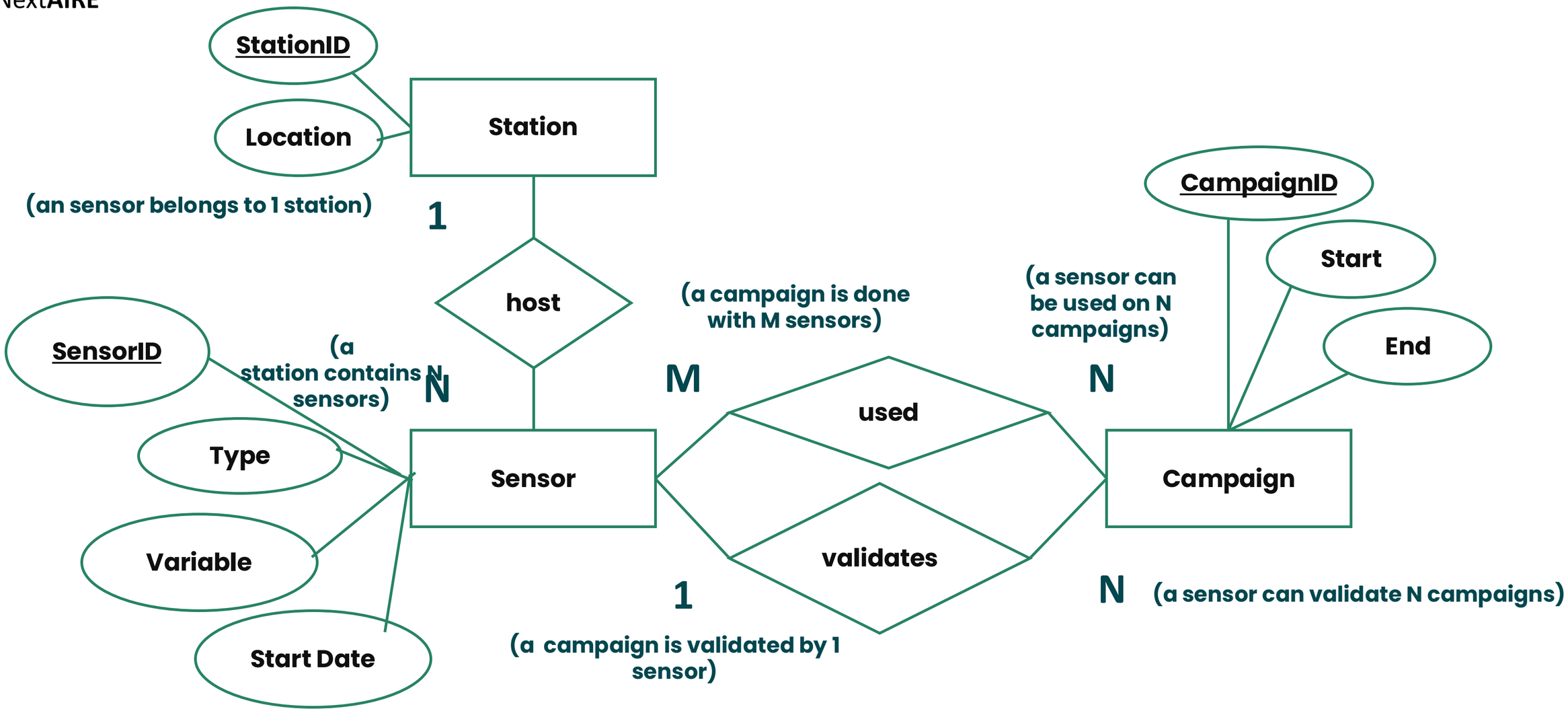
Funded by
the European Union

NextAIRE

From ER diagram to Database schema

Funded by
the European Union

Sensor database



Funded by
the European Union

Step 1: Mapping Entity Types

Each entity type directly maps to a relation:

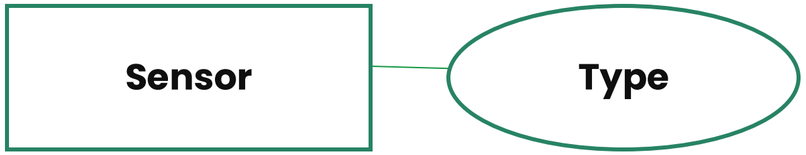
Station	**Station**(StationID: INTEGER, Location: VARCHAR(256))
Sensor	**Sensor**(SensorID: INTEGER, Type: VARCHAR(128), Variable: VARCHAR(128), StartDate: DATE)
Campaign	**Campaign**(CampaignID: INTEGER, Start: DATE, End: DATE)

Funded by
the European Union

Step 2: Mapping Attributes

Atomic attributes:

- Direct mapping of ER attributes to attributes of relation
- Choice of data types and constraints

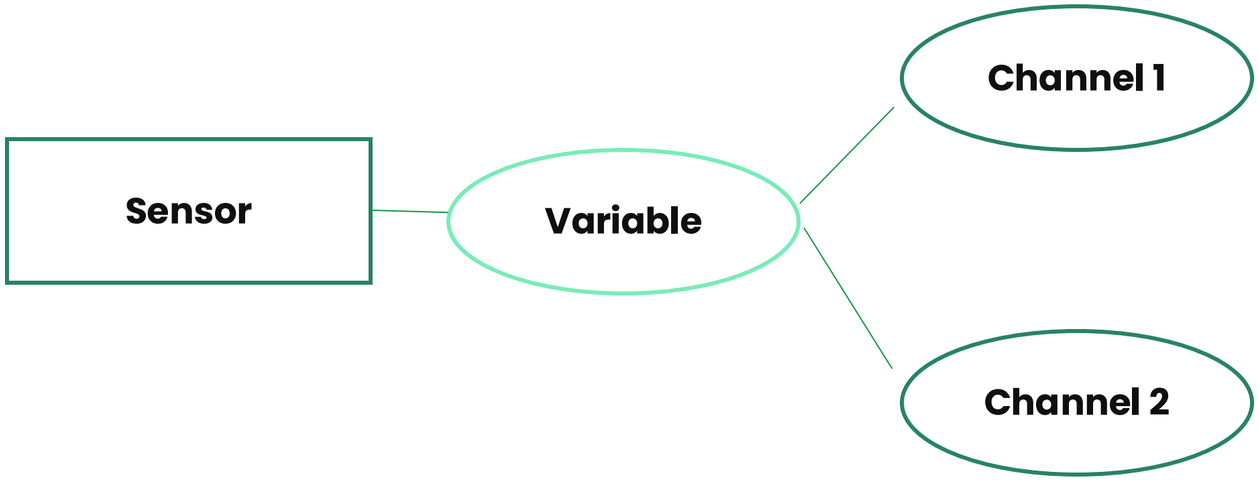


Funded by
the European Union

Step 2: Mapping Attributes

Composite attributes:

- Split into atomic attributes

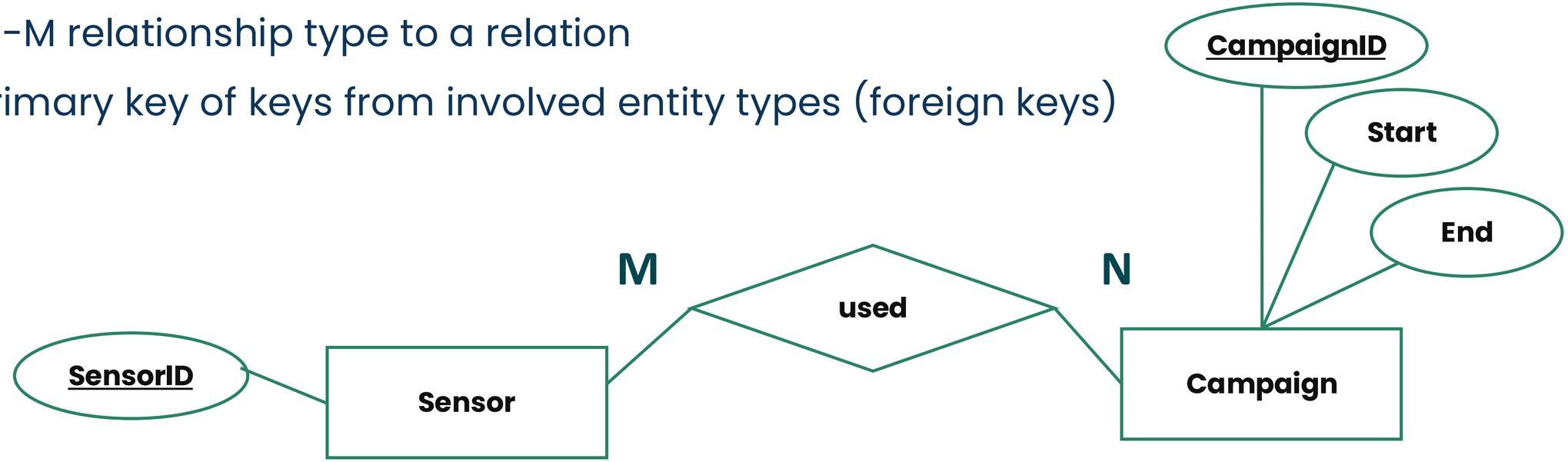


Funded by
the European Union

Step 2: Mapping Relationship Types

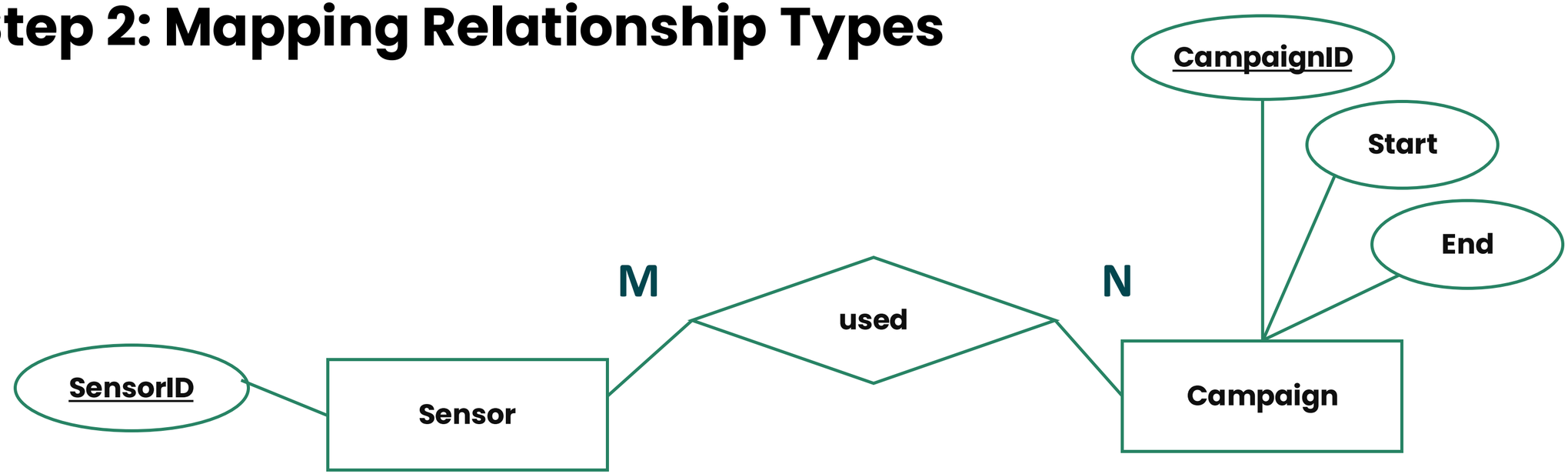
Generic Solution:

- Map every N-M relationship type to a relation
- Compose primary key of keys from involved entity types (foreign keys)



Funded by
the European Union

Step 2: Mapping Relationship Types



SensorID	CampaignID
1	12
2	12
3	01
4	07

Funded by
the European Union

NextAIRE

Structured Query Language (SQL)

Funded by
the European Union

What is a SQL Query?

```
SELECT SensorID, Type, Variable, Location
FROM Sensor, Station
WHERE Sensor.StationID = Station.StationID
AND Location LIKE 'Austria';
```

SensorID	Type	Channel_1	Channel_2	StationID
1	Sensirion SHT15	Temperature	Humidity	GRZ01
2	DHT22	Temperature	Humidity	MDZ02
3	DHT22	Temperature	Humidity	GRZ01
4	Sensirion SHT15	Temperature	Humidity	GRZ02

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

Funded by
the European Union

What is a SQL Query?

```
SELECT SensorID, Type, Channel_1, Channel_2, Location
FROM Sensor, Station
WHERE Sensor.StationID = Station.StationID
AND Location LIKE 'Austria';
```

SensorID	Type	Channel_1	Channel_2	Location
1	Sensirion SHT15	Temperature	Humidity	Austria
3	DHT22	Temperature	Humidity	Austria
4	Sensirion SHT15	Temperature	Humidity	Austria

Funded by
the European Union

Overview SQL

Structured Query Language (SQL)

- **Data Definition Language (DDL)** → manipulate the database schema
- **Data Manipulation Language (DML)** → update and query database
- **Data Control Language (DCL)** → modify permissions

Funded by
the European Union

NextAIRE

Data Definition Language

Funded by
the European Union

Create Tables

Structured Query Language (SQL)

- **Typed attributes**
- **Primary and foreign keys**
- **NOT NULL constraint**
- **→ Primary key has NOT NULL + UNIQUE constraints**

```
CREATE TABLE Sensor (
SensorID INTEGER PRIMARY KEY,
Type VARCHAR(128),
Channel_1 VARCHAR(128),
Channel_2 VARCHAR(128),
StationID VARCHAR(10),
FOREIGN KEY (StationID)
  REFERENCES Station(StationID) );
```

SensorID	Type	Channel_1	Channel_2	StationID
1	Sensirion SHT15	Temperature	Humidity	GRZ01
2	DHT22	Temperature	Humidity	MDZ02
3	DHT22	Temperature	Humidity	GRZ01
4	Sensirion SHT15	Temperature	Humidity	GRZ02

Funded by
the European Union

Create Tables

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

```
CREATE TABLE Station (
StationID CHAR(5) PRIMARY KEY,
Location VARCHAR(128) NOT NULL
);
```

Funded by
the European Union

Alter Tables

ADD/DROP columns

ALTER data type, defaults, constraints, etc

DROP TABLE → at the beginning of CREATE TABLE scripts

```
DROP TABLE IF EXIST Sensors, Stations, Campaign;
```

```
ALTER TABLE Sensor ADD Start Date DATE;
ALTER TABLE Sensor DROP Channel_2;

ALTER TABLE Campaign ADD CONSTRAINT
PRIMARY KEY (CampaignID);
```

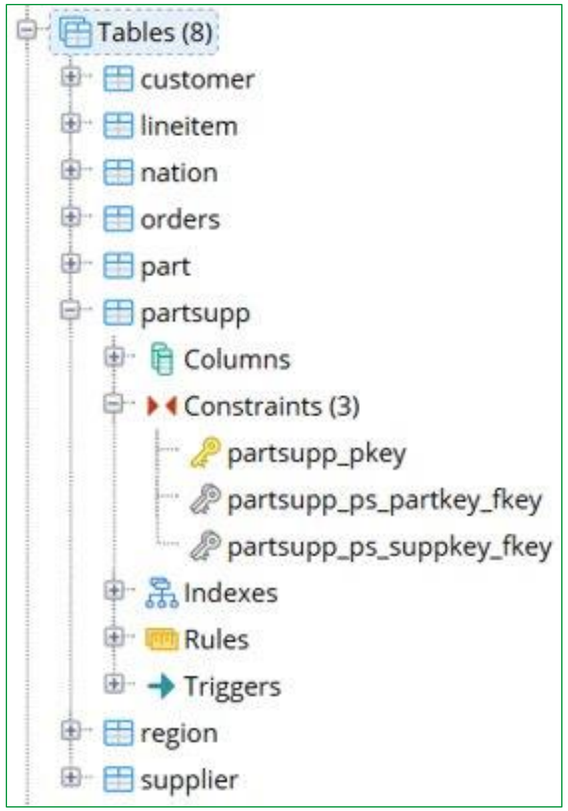
Funded by
the European Union

Database Catalog

Catalog Overview

- **Metadata** of all database objects (e.g., tables, constraints) → **read-only**
- **Accessible through SQL**
- Organized by schemas

```
SELECT * FROM information_schema.tables;
```



Funded by
the European Union

NextAIRE

Data Manipulation Language

Funded by
the European Union

Insert Data

INSERT tuple

- **Single tuple with implicit or explicit attribute assignment**
- **No need to specify attributes when you add values for all attributes**

```
INSERT INTO Sensor (SensorID, Type, Channel_1, Channel_2, StationID)
VALUES (1, 'Sensirion SHT15', 'Temperature', 'Humidity', 'GRZ01'); --Explicit
```

```
INSERT INTO Sensor VALUES (1, 'Sensirion SHT15', 'Temperature', 'Humidity', 'GRZ01'); --Implicit
```

```
INSERT INTO Sensor
VALUES
(1, 'Sensirion SHT15', 'Temperature', 'Humidity', 'GRZ01'), --Multiple tuples
(2, 'DHT22', 'Temperature', 'Humidity', 'MDZ02'),
(3, 'DHT22', 'Temperature', 'Humidity', 'GRZ01');
```

Funded by
the European Union

Insert Data

Insert from CSV file

Read and insert from a file-name using **COPY**

- Attributes in the CSV file need to match the DB table
- file-name is the path e.g., 'C:\Users\UserName\FolderName\file.csv'
- Arguments: **DELIMITER, CSV, HEADER (file with a header)**

```
COPY table-name FROM file-name CSV HEADER DELIMITER ',';
VALUES
/* this is a comment to document what is happening across multiple lines – make sure that user
“everyone” from Windows has read access to the files */
```

Funded by
the European Union

Update and Delete

UPDATE Tuple/Table

- **Update of attributes**
- **Without WHERE all tuples are updated!**

DELETE Tuple/Table

- **Delete of tuples**
- **Without WHERE all tuples are deleted!**

```
UPDATE Sensor
SET Type = 'DHT22';
WHERE SensorID = 1;

DELETE FROM Sensor
WHERE SensorID = 1;
```

Funded by
the European Union

Basic Queries

```
SELECT [DISTINCT] <column_list>
  FROM [<table_list> |
    <table1> [RIGHT | LEFT | FULL] JOIN
    <table2> ON <condition>]
  [WHERE <predicate>]
  [GROUP BY <column_list>]
    [HAVING <grouping predicate>]
  [ORDER BY <column_list> [ASC | DESC]];
```

Funded by
the European Union

Basic Queries: SELECT

Distinct and All

- Distinct = No duplicates
- All = Duplicates included

```
SELECT DISTINCT ID, Type
  FROM Sensor;
```

Aliases

- Give table or column another name
- Can then be used in the query (e.g. S.Channel_1)

```
SELECT DISTINCT Type AS T,
  FROM Sensor AS S;
```

Select * → all columns

Funded by
the European Union

Basic Queries: FROM – INNER JOIN

Specifies the tables

- Possible to combine rows from tables with related columns using JOINS
- Used for foreign key relationships

```
SELECT DISTINCT ID, Type
FROM Sensor;
```

(Inner) Join

- Get Sensor ID, Sensor Type and Station Location of all sensors

```
SELECT S.SensorID, S.Type, T.Location
FROM Sensor AS S;
INNER JOIN Station AS T
ON S.StationID = T.StationID:
```

Funded by
the European Union

Basic Queries: FROM – INNER JOIN

```
SELECT S.SensorID, S.Type, T.Location
FROM Sensor AS S;
INNER JOIN Station AS T
ON S.StationID = T.StationID:
```

SensorID	Type	Channel_1	Channel_2	StationID
1	Sensirion SHT15	Temperature	Humidity	GRZ01
2	DHT22	Temperature	Humidity	MDZ02
3	DHT22	Temperature	Humidity	GRZ01
4	Sensirion SHT15	Temperature	Humidity	GRZ02

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

SensorID	Type	Location
1	Sensirion SHT15	Austria
3	DHT22	Austria
4	Sensirion SHT15	Austria

Funded by
the European Union

Basic Queries: FROM – INNER JOIN

```
SELECT S.SensorID, S.Type, T.Location
FROM Sensor AS S;
INNER JOIN Station AS T
ON S.StationID = T.StationID:
```

SensorID	Type	Channel_1	Channel_2	StationID
1	Sensirion SHT15	Temperature	Humidity	GRZ01
2	**DHT22**	**Temperature**	**Humidity**	**MDZ02**
3	DHT22	Temperature	Humidity	GRZ01
4	Sensirion SHT15	Temperature	Humidity	GRZ02
5	**Sensirion SHT71**	**Temperature**	**Humidity**	**NULL**

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

Sensor 2 → FK not found in Station

NULLS → Omitted in output

SensorID	Type	Location
1	Sensirion SHT15	Austria
3	DHT22	Austria
4	Sensirion SHT15	Austria

Funded by
the European Union

Basic Queries: FROM – OUTER JOIN

```
SELECT S.SensorID, S.Type, T.Location
FROM Sensor AS S;
LEFT OUTER JOIN Station AS T
ON S.StationID = T.StationID:
```

SensorID	Type	Channel_1	Channel_2	StationID
1	Sensirion SHT15	Temperature	Humidity	GRZ01
2	**DHT22**	**Temperature**	**Humidity**	**MDZ02**
3	DHT22	Temperature	Humidity	GRZ01
4	Sensirion SHT15	Temperature	Humidity	GRZ02
5	**Sensirion SHT71**	**Temperature**	**Humidity**	**NULL**

StationID	Location
GRZ01	Austria
GRZ02	Austria
MDZ01	Argentina
EZE01	Argentina

All non-matching fields **(in LEFT)** are filled with NULL **(from RIGHT)**

SensorID	Type	Location
1	Sensirion SHT15	Austria
2	**DHT22**	**NULL**
3	DHT22	Austria
4	Sensirion SHT15	Austria
5	**Sensirion SHT71**	**NULL**

Funded by the European Union

Basic Queries: WHERE

To add conditions to the query

- Combine conditions with AND / OR
- WHERE NOT to negate conditions
- Single quotes for text fields
- Wild cards for LIKE (% placeholder for an char)
- **Full list of operators:**

  https://www.w3schools.com/sql/sql_where.asp

```
SELECT SensorID, Type, Channel_1, Channel_2,
StationID
FROM Sensor
    WHERE StationID = 'GRZ01' OR StationID = 'GRZ02';
```

```
SELECT SensorID, Type, Channel_1, Channel_2,
StationID
FROM Sensor
    WHERE Type LIKE 'Sensirion%';
```

Funded by
the European Union

Grouping and Aggregation

GROUP BY

- Allows to summarize data by categories
- Summarize → aggregates (MAX, MIN, AVG, COUNT)

```
SELECT StationID, COUNT(*) AS NumberOfSensors
FROM Sensor
GROUP BY StationID;
```

How many sensors are installed at each station?

Funded by
the European Union

Grouping and Aggregation

HAVING

- Filter groups after calculation of aggregates

```
SELECT StationID, COUNT(*) AS NumberOfSensors
FROM Sensor
GROUP BY StationID
HAVING COUNT (*) > 1;
```

Which stations have more than one sensor?

Funded by
the European Union

Sorting

Sorting

- Convert output into a sorted list of tuples
- **DESC** → from highest value to lowest
- **ASC** → from lowest value to highest
- Multiple fields possible
- Order by first and in case of ties by the second, etc.
- Evaluated last in a query

```
SELECT SensorID, Type, Channel_1, Channel_2,
StationID
FROM Sensor
ORDER BY StationID ASC, Type ASC;
```

Funded by
the European Union

NextAIRE

Data Control Language

Funded by
the European Union

Data Control Language

Set access permissions on tables

- **GRANT** query/modification rights on database objects for specific users, roles
- **REVOKE** access rights from users, roles
- Same syntax for **UPDATE**, **INSERT**, etc

```
GRANT SELECT
ON TABLE Sensor
TO liacono;
```

```
REVOKE SELECT
ON TABLE Sensor
TO liacono;
```

Funded by
the European Union

Thank You!

Lucas Iacono

Know Center GmbH

liacono@know-center.at

www.know-center.at

NextAIRE

Funded by
the European Union